Quant: A Post-Quantum Peer-to-Peer Electronic Cash System

The Quant contributors
github.com/tehrealaju-svg/quant-core · October 2026 · version 2

Abstract

Bitcoin showed that people can send money to each other without a bank, but every coin it protects is guarded by a signature that a large enough quantum computer could forge. Quant is a cash system built from the start on cryptography that is believed to survive quantum computers: Falcon-512 signatures, chosen by the U.S. National Institute of Standards and Technology for exactly this purpose, and BLAKE3 hashing for proof-of-work. Post-quantum signatures are larger than the ones they replace, so Quant is designed to stay small anyway: signatures are thrown away once they are buried a week deep, and every two weeks the whole state of the ledger is written down, fingerprinted inside a block, and handed to new users so they do not need the full history. The same chain carries user-created tokens and a built-in exchange, so trading needs no company in the middle. There is no premine, no developer fee, and fees are a fraction of a cent.

1. Introduction

Digital cash has two hard problems: proving who owns a coin, and agreeing on the order of payments so nobody can spend the same coin twice. Bitcoin solved both with digital signatures and a chain of proof-of-work [1]. The second idea holds up well against quantum computers. The first does not. Bitcoin's signatures (ECDSA and Schnorr on the secp256k1 curve) rest on a math problem that Shor's algorithm solves efficiently on a quantum computer [2]. Once such a machine exists, anyone who can see a public key can compute the private key behind it.

Quant keeps Bitcoin's overall design, because it is proven and widely understood, and replaces the parts that age badly. Ownership is proven with lattice-based signatures. The chain is kept small with pruning and snapshots. Tokens and trading are part of the protocol, so the common things people want to do with a cryptocurrency do not depend on a trusted operator.

2. Transactions

A coin is a chain of signatures, as in Bitcoin. Each transaction spends earlier outputs and creates new ones. An output names an amount, an asset (QNT or a token), and an address. An address is the BLAKE3 hash of a Falcon public key, so the public key itself stays hidden until its owner spends. To spend, the owner reveals the key and signs the hash of the whole transaction, bound to this network's genesis block so it cannot be replayed on another chain.

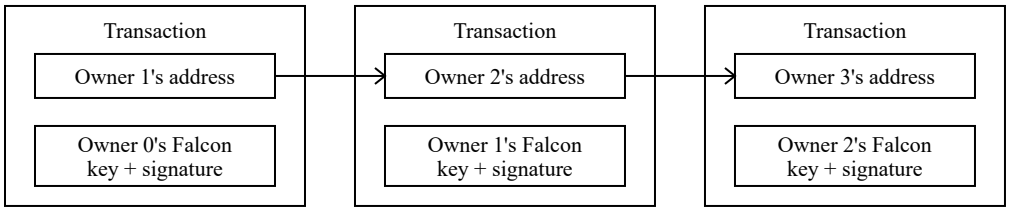


Figure 1. Each owner passes a coin on by signing it over to the next owner's address.

Several coins sent to the same address need only one signature: later inputs point back to the first one. Every number is stored in a compact variable-length form with exactly one valid encoding, so a transaction's id cannot be changed by re-encoding it.

3. Quantum Security

Quant signs with Falcon-512, standardized by NIST as FN-DSA [3]. Its security rests on finding short vectors in structured lattices, a problem for which no efficient quantum algorithm is known. Falcon was chosen over the other NIST schemes because its signatures are the smallest: about 655 bytes, with an 897-byte public key, against roughly 2,400 bytes for ML-DSA. Hiding the key behind a hash adds a second layer: an attacker cannot even begin until the owner spends.

Hash functions are only weakened, not broken, by quantum computers. Grover's algorithm [4] at best halves the effective strength of a hash, so 256-bit BLAKE3 [5] keeps about 128 bits of security for addresses and block ids. Proof-of-work is affected the same way: a quantum miner gets a square-root speedup on a fixed problem, which the difficulty adjustment absorbs like any other faster hardware.

4. Proof-of-Work and the Chain

Blocks form a chain as in Bitcoin: each 152-byte header contains the hash of the previous one, a Merkle root of its transactions, a second Merkle root of their signatures, a state commitment (Section 6), a time, a difficulty target and a nonce. Miners search for a nonce that makes the BLAKE3 hash of the header fall below the target. BLAKE3 is fast on ordinary computers, phones and small boards, so anyone can verify the chain cheaply. The target is recalculated every block from the last 60 blocks (the LWMA method [6]) to keep blocks two minutes apart, so a sudden drop in mining power cannot stall the chain for weeks. Nodes follow the chain with the most total work, and refuse to reorganize more than one week of blocks, which also protects new nodes from long, cheaply faked histories.

5. Tokens and the Built-In Exchange

Anyone can create a token, such as a meme coin, with a single transaction. It costs nothing but the normal fee. The creator picks a ticker, a name, a number of decimals and a supply, and the whole supply appears in their wallet. The token's id is derived from the coin the transaction spends, so ids can never collide. After that, tokens move exactly like QNT.

Trading happens against on-chain pools, one per token, each holding QNT and the token. Prices follow the constant-product rule used by Uniswap [7]: a pool holding x QNT and y tokens only accepts trades that keep the product from falling.

$$(x + 0.997 \cdot \Delta x) \cdot (y - \Delta y) \geq x \cdot y$$

A trade states exactly what it pays in and what it takes out, and the network checks the result against the rule above, so a trader can never receive less than they signed for. If the price moves first, the trade is rejected and no money is lost. The 0.3% fee stays in the pool and belongs entirely to the people who supplied its liquidity. Nobody else takes a cut. Every asset in every transaction must balance exactly, so tokens cannot be created from nothing.

6. Keeping the Chain Small

Post-quantum signatures are large, and they are most of a transaction. But a signature is only needed once: to check that a spend was authorized. Quant keeps signatures in a separate part of each block with its own Merkle root. Nodes check them, keep them for one week (5,040 blocks) so they can survive a reorganization, and then delete them. What remains of a typical payment is about 117 bytes, smaller than a Bitcoin payment.

History is the other cost. Every 10,080 blocks (about two weeks) each node writes down the full state of the ledger: every unspent coin, every token and every pool. It hashes this snapshot, and the block 100 blocks later must carry that hash in its header. A miner who lies about the state produces a block every honest node

rejects. A new node downloads the headers, checks the proof-of-work, and then fetches the newest snapshot from any peer and checks it against the hash in the header. From there it verifies only the blocks after the snapshot.

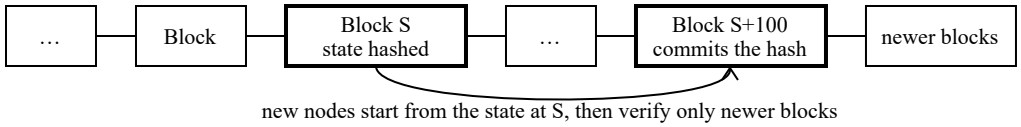


Figure 2. A state snapshot is committed in a later header, so new nodes can skip old history.

7. Supply

New coins are created only by mining. There is no premine, no developer fee, and the genesis block pays nobody. Instead of Bitcoin's sudden halvings, the reward shrinks a little every block: it is $1/2^{23}$ of the coins left on a 75-million curve, starting near 8.9 QNT. After about 30 years, when 45.6 million QNT exist, the reward stops shrinking and stays at 3.5 QNT per block forever. This tail keeps miners paid without relying only on fees. It also means people who arrive later can still earn new coins, and the yearly increase keeps falling as a share of the total. QNT has 10 decimal places.

Years after launch	Block reward (QNT)	Total supply (QNT)	Yearly growth
0	8.94	0	–
1	8.66	2.31 M	–
5	7.64	10.88 M	–
10	6.53	20.18 M	–
20	4.78	34.94 M	–
30 (tail begins)	3.5	45.72 M	2.0%
60	3.5	73.33 M	1.3%
100	3.5	110.15 M	0.8%

8. Fees

The minimum fee is 0.00001 QNT per 1,000 bytes, about 0.00002 QNT for a normal payment. Fees go to the miner of the block, and nowhere else. Fees are charged on size, not value, so sending a large amount costs the same as sending a small one. The smallest QNT output is 0.000001 QNT, which stops the ledger from filling up with worthless dust.

9. Network

Nodes find each other without any central server: through the public BitTorrent DHT, broadcasts on the local network, and addresses saved from earlier sessions. IPv6 is supported, so nodes behind carrier-grade NAT, as on many mobile and satellite connections, can still be reached. Phone wallets do not need the whole chain. As in Bitcoin's simplified payment verification [1], they download only block headers and ask full nodes for their own coins, each with a Merkle proof that it is really in a block. Browser wallets and websites build and sign transactions locally and send only the signed result, so private keys never leave the user's device.

10. Security

As in Bitcoin, the chain is safe as long as honest miners control most of the mining power. An attacker's chance of catching up from z blocks behind falls off exponentially with z [1]. With two-minute blocks, waiting 30 confirmations gives about the same safety, in the same hour, as six Bitcoin confirmations. Pruning removes signatures only after a week, beyond the reorganization limit, and a snapshot is trusted only when buried under real proof-of-work. The main new risk is cryptographic. Falcon is younger than ECDSA, so the format carries version numbers that let the network add another signature scheme by soft fork if lattices are ever weakened.

11. Conclusion

Quant keeps what works in Bitcoin, a public chain secured by proof-of-work and coins controlled only by their owners' keys, and replaces what a quantum computer would break. Falcon-512 signatures protect ownership. Pruning and committed snapshots keep the chain small enough that ordinary computers can run full nodes. Tokens and a constant-product exchange are part of the protocol, so everyday uses need no trusted company. The supply is fixed by code, fair from the first block, and pays miners forever without a developer tax.

References

1. S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
2. P. W. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," *SIAM J. Computing*, 1997.
3. P.-A. Fouque et al., "Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU," NIST PQC submission; standardized as FN-DSA (FIPS 206).
4. L. K. Grover, "A fast quantum mechanical algorithm for database search," *STOC*, 1996.
5. J. O'Connor, J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn, "BLAKE3: one function, fast everywhere," 2020.
6. zawy12, "LWMA difficulty algorithm," github.com/zawy12/difficulty-algorithms, 2018.
7. H. Adams, N. Zinsmeister, D. Robinson, "Uniswap v2 Core," 2020.